



WX Claude Code

Conversão governada de projetos WINDEV, WEBDEV e WINDEV Mobile para outra plataforma. Questionário, gates com aprovação humana, equipe WLanguage sobre o Help, PMO com Scrum, Kanban e PDCA, e o contexto da primeira sessão do Claude Code gerado das respostas.

VERSÃO 3.20.0 · MEDIDO EM 2026-09-04

Em números

94

AGENTES

10 + 40

PAPÉIS COM PDCA

44

TESTES DE REGRESSÃO

24

SCRIPTS PYTHON

14

BLOCOS DO QUESTIONÁRIO

36

PROVAS EM SESSÃO REAL

24

CENAS NO VÍDEO

12.583

LINHAS DE PYTHON

Cada número é contado no repositório por `docs/dossie/numeros-do-plugin.py`; a tabela completa está em `docs/relatorio-do-plugin.md`.

Como um projeto atravessa o plugin

Questionário

bloco 0, A–L

Pré-flight G0

anexos conferidos

Inventário G1-G3

matriz, decisões

Piloto G4

golden master

Ondas G5-G6

papéis A–J, PDCA

Cutover G7

relatório, exportação

O questionário tem o bloco 0 da empresa (16 itens), as letras A a J, K do ambiente (8 itens) e L do contexto (6 itens); a letra F sozinha tem 14 subperguntas de qualidade de ERP. Dele saem até 101 arquivos: manifesto, configuração, respostas legíveis, empresa, processo, ambiente com instalador e SQL dos papéis, prompts de kickoff e prototipação, `INDEX_FILES.md`, o `.claude/` do projeto, `Dockerfile` e `compose`. O PMO fecha cada sprint com o relatório de onze seções e a exportação entrega o projeto organizado, com hashes, na pasta do usuário.

Regras que o projeto aprendeu medindo

Senha nunca em texto puro

o questionário guarda só o nome da variável; o gerador recusa chave senha/token com valor e valores com formato de token; a sessão real com a senha colada não a repetiu.

Valor de questionário é vetor de injeção

tudo que vira bash, SQL, YAML ou JSON passa por validação antes de gravar; identificador é identificador, porta é inteiro; teste com onze injeções.

Número visível sai de um gerador

os deste dossiê vêm de numeros-do-plugin.py; a página para investidores já ficou quatro versões com 34 agentes e 12 testes por ter sido digitada.

Interface só se prova exercitando

cada funcionalidade nova teve sessão real gravada; uma delas achou o .env.exemplo caindo no filtro de .env, outra o manifesto dizendo missing para dados que existiam.

O que não se mede fica INDISPONÍVEL

painel, relatório e medidor de ambiente nunca mostram zero no lugar do desconhecido.

Guarda nova entra pedida, não imposta

a licença trava só os scripts e a escrita em .wx-migration; o resto do Claude Code segue; o hook custa 54 ms medidos.

Skill que não aparece na listagem não existe.

As oito skills de ERP vieram com descrições de até 900 caracteres, e a impeccable já tinha provado que 895 some da listagem de uma sessão nova. Entraram com 150, os originais guardados ao lado; a prova é a sessão listar as onze pelo nome.

O que foi provado em sessão real

Cada linha é um print em docs/prints/: saída real de uma sessão do Claude Code ou de um script, sem edição. O vídeo de uso (3 min 07 s, 24 cenas) reproduz as mesmas capturas.

PRINT	ORIGEM
01	`claude plugin validate` no plugin e no marketplace, mais o validador offline em modo estrito
02	sessão `-p` pedindo a lista de skills e agentes com prefixo `wx-claude-code:`
03	três turnos reais (`claude -p` e `-c`): pergunta A, resposta, confirmação e pergunta B, resposta «não tenho», pergunta C

PRINT	ORIGEM
04	`aplicar_questionario.py` sobre respostas de exemplo e o `wx_preflight.py` real (BLOCKED por PDF de mentira, como devia)
05	o `DESIGN.md` que a letra F gera
06	`query_wlanguage_help.py --verify` e `--query HReadSeekFirst`
07	`claude -p "/wx-claude-code:laudo-tokens fase-1"` com fonte de sessões marcada INDISPONÍVEL
08	`pmo.py iniciar/gastar/status` e `rotear_modelo.py` com subida, rebaixamento a 85 %, bloqueio a 105 % e fallback
09	sessão `-p` com `Agent` liberado: dois símbolos delegados aos `wl-hfsql-specialist` e `wl-communication-specialist`, com o member do Help de cada um
10	`pmo.py sprint abrir`, dois ciclos `pdca` (um infrutífero, recusado sem `--proxima`), `kanban` com WIP e `sprint fechar` com resumo de doze seções
11	sessão `-p` com `Agent` liberado: `/wx-claude-code:pmo status` delegado ao agente do PMO, que rodou os scripts e apontou a dessincronia entre plano e sprint
12	o painel gerado sozinho por `pmo.py sprint fechar` num projeto com bloco 0, lacunas, decisões, riscos, PDCA e roteamento: o relatório de onze seções, o kanban e a base, aberto no Chromium com `prefers-color-scheme: light`
13	sessão `-p` e `-c`: letra H com «não sei a linguagem», sinais, três opções com a recomendada primeiro
14	o exemplo `estoque-wx`: `aplicar_questionario.py`, G0 `CONDITIONAL` sem erros, `extrair_pdf.py`, `golden.py` 9/10
15	`sprint abrir` com `--item ID:PAPEL`, `kanban` com `[C dba]` etc., `sprint fechar` e `entregar` listando o zip; `tecnicas-aplicadas.md` de dentro do zip
16	`aplicar_questionario.py` no exemplo com as treze subperguntas de F; trechos do `DESIGN.md` (grids, tabela de botões, posição, fundo) e do `PRODUCT.md`
17	três turnos reais: abertura do wizard com o item 0.1 sozinho, resposta, confirmação e 0.2, resposta e 0.3
18	sessão limpa em que o usuário cola a senha do GitHub no meio da resposta: o agente não a reproduz de forma nenhuma, pede para revogar e registra só URL e usuário (a senha digitada foi mascarada no print; a saída do agente é a real, e o `grep` pelo valor devolve 0)

PRINT	ORIGEM
19	sessão <code>`-p`</code> e <code>`-c`</code> com <code>`--allowedTools Read`</code> : letra H com os quatro sinais, três opções com a recomendada primeiro, e o processo de conversão para Python lido de <code>`references/perfis-de-destino.md`</code> , peça por peça
20	sessão <code>`-p`</code> e <code>`-c`</code> com leitura liberada: letra F pede a tela principal como modelo (F0); o agente abriu as capturas de <code>`inputs/screenshots/`</code> , propôs o que preservar a partir do que viu, registrou F0 e só então passou à F1
21	<code>`licenca.py verificar`</code> sem serial, sessão <code>`-p`</code> do PMO recusando pelo contexto do hook <code>`SessionStart`</code> , <code>`licenca.py instalar`</code> com um serial de demonstração e a mesma sessão rodando o PMO em seguida
22	sessão nova, sem contexto, com leitura liberada: perguntada sobre o aprovador e o prazo, achou os dois em <code>`respostas_questionario.md`</code> pelo <code>`CLAUDE.md`</code> , sem perguntar de volta
23	sessão limpa em que o usuário adianta a letra K2 com a senha do root do PostgreSQL: o agente não a reproduz de forma nenhuma, registra banco, superusuário e papéis, e diz que vai pedir só o nome da variável (a senha digitada foi mascarada no print; o <code>`grep`</code> pelo valor na saída do agente devolve 0)
24	dois turnos reais (<code>`-p`</code> e <code>`-c`</code>): K7 «sim» ao n8n, o item K7.1 sozinho, resposta, e o K7.2 avisando que sem K2 o banco do n8n pode ser SQLite ou PostgreSQL
25	sessão nova com leitura liberada, num projeto recém-aplicado: perguntada o que ler e qual o escopo da v1, listou a ordem de leitura, os cinco requisitos do kickoff, a estratégia, e recusou escrever código por falta de G0; de quebra achou um conflito real entre os anexos e o manifesto
26	sessão <code>`-p`</code> com <code>`Agent`</code> e <code>`Bash`</code> : <code>`/wx-claude-code:pmo exportar`</code> grava o projeto organizado na pasta pedida e explica o que ficou de fora; achou um defeito real (o <code>`env.exemplo`</code> caía no filtro de <code>`env`</code>), corrigido com teste
27	sessão <code>`-p`</code> num projeto com log antigo: o hook <code>`SessionStart`</code> rodou o zelador e a sessão mostra o registro com bytes medidos
28	dois turnos reais (<code>`-p`</code> e <code>`-c`</code>): a resposta abre com a identificação <code>`Bloco0001-SP00002-Análise da base de dados`</code> · <code>`data`</code> injetada pelo hook; no segundo turno o agente se recusa a inventar o objetivo da sprint nova e antecipa a identificação <code>`SP00003`</code>
29	sessão real: perguntada qual agente entra num PDCA infrutífero, apontou o Pesquisador (equipe B), achou o pedido pendente em <code>`pmo/pesquisas.md`</code> com a hipótese morta e a próxima, e deu o status por agente recusando inventar estado para quem não registrou
30	duas sessões reais: a pergunta sobre a estratégia foi respondida citando <code>`processo-de-conversao.md#L5-L9`</code> , o localizador que o RAG injeta; pedido para apagar um anexo, o agente recusou pela regra antes de o hook precisar negar (o hook está provado por teste, com <code>`stdin`</code>)

PRINT	ORIGEM
31	sessão real num projeto recém-aplicado: perguntada o que faz <code>`HReadSeekFirst`</code> , consultou o Help pelo tema que o hook do RAG apontou, citou a página com id e hash, e separou semântica técnica de regra de negócio, como o <code>`CLAUDE.md`</code> gerado manda
32	<code>`claude --plugin-dir wx-claude-code -p "liste as skills"`</code> (skills globais do ambiente omitidas do print): as onze skills do plugin aparecem, as oito de ERP com descrição de até 150 caracteres; perguntada por partidas dobradas e estorno, apontou <code>`erp-accounting`</code>
33	sessão real num projeto com L6 = sim: leu <code>`CLAUDE.md`</code> e <code>`AGENTS.md`</code> , listou módulo → skill, resumiu a ADR 0002 (sem multiempresa na v1) e carregou a <code>`erp-inventory`</code> , citando INV-01 e INV-06 sobre saldo
34	sessão real na 3.18.0: leu <code>`docs/skills-recomendadas.md`</code> e separou as que cabem (Rust, PostgreSQL, React) das que ficaram fora (Supabase não instalado, sem multiempresa); citou <code>`NUMERIC(19,4)`</code> e <code>`TIMESTAMPTZ`</code> do modelo de dados; explicou que cada mitigação STRIDE vira <code>`SEC-*`</code> e teste
35	sessão real na 3.19.0: leu <code>`artefatos/CATALOGO.md`</code> e o <code>`CLAUDE.md`</code> , distinguiu o artefato arquivado dos três só declarados e recusou usar o que não foi submetido; explicou o hash como prova; carregou a skill de PHP e citou ponto flutuante em dinheiro e comparação frouxa, com <code>`BigDecimal`/centavos</code> e <code>`DECIMAL(19,4)`</code>
36	sessão real na 3.20.0: listou os dezessete comandos, rodou o <code>`listar_perguntas.py`</code> (59 perguntas, aprovador 0.16, tela modelo F0), e respondeu que legado só PHP com destino Elixir é aceito — dizendo como preencher e que o caso pétreo continua sendo WLanguage para outra linguagem

O que ainda não foi provado

- Nenhum projeto WINDEV real passou pelos gates G1 a G7 de ponta a ponta; o exemplo ESTOQUE é sintético.
- Os scripts de ambiente são bash; não há PowerShell, e o público usa Windows.
- A licença é dissuasão por hook; servir corpus e agentes de um servidor ficou para depois, por decisão do dono.
- O custo em tokens do questionário inteiro numa sessão real não foi medido.
- O questionário não pausa nem retoma; com mais de setenta itens, isso pesa.

Versões

VERSÃO	DATA	O QUE ENTROU
3.20.0	2026-09-04	legado E/OU, destino em qualquer linguagem, e um comando para cada recurso
3.19.0	2026-09-04	PHP nos dois sentidos e o bloco M dos artefatos submetidos
3.18.0	2026-09-04	o que a pesquisa no skills.sh acrescentou ao esqueleto, e por que as 27 skills não entram no plugin
3.17.0	2026-09-04	vídeo com a cena 24 (esqueleto de ERP) e dossiê com os números medidos
3.17.0	2026-09-04	oito skills de ERP e o esqueleto de projeto que o questionário gera (L6)
3.16.1	2026-09-04	o corpus WLanguage entra no CLAUDE.md gerado e no RAG por símbolo
3.16.0	2026-09-04	hooks que fazem valer as regras e um RAG local do projeto
3.15.0	2026-09-04	equipe prioritária de dez agentes, cada um com gatilho próprio
3.14.0	2026-09-03	blocos e sprints numerados, identificação obrigatória em cada interação, sprint zipada
3.13.0	2026-09-03	exportar o projeto organizado, zelador diário, relatório e dossiê medidos, vídeo com todas as provas
3.12.1	2026-09-03	revisão inteira — injeção no gerador, filtro de segredos, portão G0 e 50 descrições cortadas
3.12.0	2026-09-03	letra L — o contexto da primeira sessão sai pronto do questionário
3.11.0	2026-09-03	n8n integrado ao projeto (K7) e privilégios do instalador (K0)
3.10.0	2026-09-03	letra K — Rust/Cargo, PostgreSQL, MySQL, MariaDB, Supabase e GitHub no questionário

VERSÃO	DATA	O QUE ENTROU
3.9.0	2026-09-03	o wizard pergunta o aprovador e grava todas as respostas em respostas_questionario.md
3.8.1	2026-09-03	relatório de onze seções gerado sozinho ao fechar sprint e na entrega
3.8.0	2026-09-03	serial de ativação assinado com RSA, hooks de licença e marca d'água
3.7.2	2026-09-03	a tela principal do projeto entra como modelo visual (F0)
3.7.1	2026-09-03	H e I mostram o processo de conversão e perguntam a estratégia
3.7.0	2026-09-03	bloco 0 do questionário — empresa, projeto, prazo, orçamento e GitHub sem senha
3.6.1	2026-09-03	botões perguntados um a um — vocabulário, posição, ícone, cor e fundo
3.6.0	2026-09-03	a letra F vira oito subperguntas de ERP, cada uma ligada a um comando do Impeccable
3.5.0	2026-09-03	dez papéis com PDCA, backlog com dono e entrega zipada ao stakeholder
3.4.0	2026-09-03	para qual linguagem converter, WL_C# como perfil, prints e vídeo com o exemplo
3.3.0	2026-09-03	projeto de exemplo real, provas em código, portão G0 e testes de regressão
3.2.1	2026-09-03	pacote com o questionário letra a letra

Lido do `git log`: só os commits cuja mensagem começa pela versão.

Onde está cada coisa

ARQUIVO	O QUE É
MANUAL.md, docs/manual-de-uso.pdf	manual de oito capítulos
docs/relatorio-do-plugin.md	relatório medido do estado atual
docs/investidor/	página e PDF para investidores
docs/prints/, docs/video/	provas reais e o vídeo de uso
docs/analise-aula-vibe-coding.md	o que a aula de vibe coding ensinou e o que não se copiou
docs/telas-licenca/	sete telas do fluxo de liberação de licença
licenca/LEIA-ME.md	o que o serial protege e o que não
skills/LEIA-ME-erp.md	as oito skills de ERP do pacote skills.sh e o esqueleto que L6 gera
exemplos/estoque-wx/	projeto de exemplo e teste de regressão do fluxo inteiro
tests/testes.py	44 testes; o validador estrito os roda

Gerado por `docs/dossie/gerar-dossie.py` em 2026-09-04; regenerar em vez de editar. Built to store. Engineered to scale.